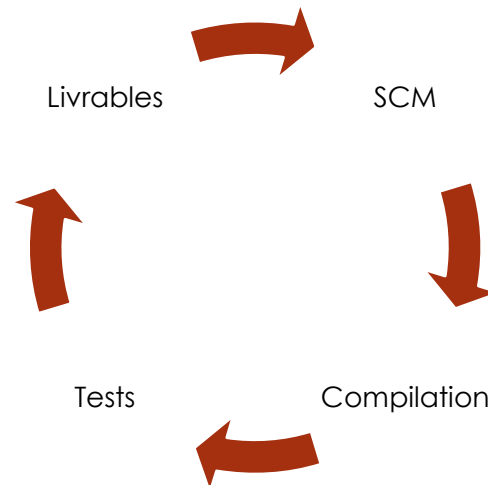


Intégration Continue

1



Objectifs de la formation

- Comprendre l'intégration continue : ses objectifs et ses enjeux
- Maîtriser les bonnes pratiques de l'intégration continue
- Savoir mettre en pratique l'intégration continue

Définition

- **L'intégration continue** est un ensemble de pratiques utilisées en génie logiciel, consistant à vérifier, à chaque modification de code source que le résultat des modifications ne produit pas de régression dans l'application développée.
- Le concept a pour la première fois été mentionné par Grady Booch et se réfère généralement à la pratique de l'extreme programming (XP)



Indispensable

- Pour faire de l'IC il faut **OBLIGATOIREMENT**
 - Du code (évidence ...)
 - Un outil de gestion de source (= **Source Control Manager**)
 - ❑ Git, SVN, CVS, ...
 - Des tests Unitaires
 - ❑ Et / ou d'intégrations
 - Une mécanique automatisée / scriptée de Build
 - ❑ A travers un outil : Maven, NPM, Make file, ...
 - Un ordonnanceur de build
 - ❑ A travers un outil : Jenkins, Gitlab CI, Travis, Hudson, CRON ...

Optionnel

- Pour faire de l'IC il est conseillé d'avoir aussi
 - Une mécanique d'analyse qualité
 - ❑ A travers un outil : Sonar, PMD, Codacy, ...
 - Une mécanique de fabrication de rapports
 - ❑ Génération de documentation sur le projet
 - ❑ Génération d'un rapport sur les usages du SCM
 - ❑ Rapport sur le respect d'une architecture donnée

Déploiement Continue

- IC débouche souvent sur la mécanique de Déploiement Continue (DC)
- Après la validation de la build, on la déploie
 - Il faut donc pour
 - ❑ Un (ou plusieurs) environnement(s) de déploiement
 - ❑ Potentiellement des tests d'intégrations

Vision Globale

➤ L'IC est un Workflow = enchainement d'étapes

1. Je fais du code
2. Je *push* mon code via mon SCM sur mon serveur
3. Mon ordonnanceur récupère mon code et le **compile**
4. Mon ordonnanceur **lance les tests Unitaires** et fabrique un compte rendu de passage de tests
5. Mon ordonnanceur **lance l'analyse qualité** (génère des rapports)
6. Mon ordonnanceur **lance la fabrication du/des livrables**

Si OK

Si OK

Si OK

Vision Globale

- L'DC est aussi un Workflow = enchainement d'étapes
1. Mon IC se termine correctement avec la fabrication de livrables
 2. Optionnel : Mon ordonnanceur fabrique un environnement cible en fonction d'un paramétrage donné
 3. Mon ordonnanceur **déploie le livrable** sur la plateforme cible
 4. Mon ordonnanceur **réalise des tests d'intégrations**
 5. Mon ordonnanceur prévient que le cycle s'est terminé avec succès, le cas échéant un supprime le déploiement et revient en arrière

Vision Globale - IC

9



Dev

1 - push



2 - build

Maven™



Build

3 - Tests

JUnit



PHPUnit

Tests Unitaires

4 - Qualité

sonarqube

Qualité

5 - Livrable

WAR,
JAR,
EXE, ...

Livrables

Etape - 0 - Installer son serveur

➤ Pour faire de l'IC je dois donc avoir une machine serveur qui abrite

➤ Un **serveur** de source (SCM)

☐ GitLab

https://en.wikipedia.org/wiki/Comparison_of_source-code-hosting_facilities

☐ BitBucket

☐ GitHub

➤ Un (ou plusieurs) outil(s) de build

☐ Maven / Ant / Gradle (Java)

https://en.wikipedia.org/wiki/List_of_build_automation_software

☐ NPM (JS)

☐ Scriptes

➤ Un **serveur / ordonnanceur / orchestrateur** de build

☐ GitLab -> GitLab CI

☐ BitBucket -> Pipeline

☐ GitHub -> Travis

https://en.wikipedia.org/wiki/Comparison_of_continuous_integration_software

☐ Jenkins / Hudson

☐ Un Cron scripté

Etape - 0

➤ Remarque

- La plus part du temps vous n'avez pas à 'installer' quoi que ce soit
- Tout (ou presque) est déjà en place via du SAS (Software as a Service)

➤ Vous devez donc vérifier que tout est disponible

- Faut-il payer ?
 - ☐ Un CI consomme des ressources (mémoire, espace disque, processeur)
- Faut-il compléter l'installation ?
- Êtes-vous limiter dans vos build ?

Etape - 0

- Vous pouvez aussi déployer votre serveur de CI via une image ou un conteneur
 - Bitnami
 - Amazon
 - ...
- Elle contiendra tout ce dont vous aurez besoin pour faire votre intégration continue.
 - Souvent payant

Etape - 1 - Push

- J'ai réalisé mon code, je suis satisfait de mon travail
- Je push sur le serveur SCM
 - Remarque : Potentiellement il y aura ici une Push Request **ou une validation humaine** faite côté serveur pour valider votre push
- Mon push est validé $\Leftrightarrow$ l'ordonnanceur de build prend la main
 - Remarque : il peut aussi s'enclencher de manière périodique = tous les soirs à minuit par exemple

Etape - 2 - Compilation

➤ L'ordonnanceur va **compiler** votre projet

- Il commence souvent par récupérer votre projet de votre SCM (il fait un git clone)
- Ensuite, il fera usage de votre mécanique de build et de vos options de build pour compiler
 - ☐ Maven / Gradle / Ant pour le Java
 - ☐ NPM pour le JS
 - ☐ Make pour le C / C ++
 - ☐ ...
- Si la compilation est ok $\Leftrightarrow$ passage à l'étape suivante
- Sinon $\Leftrightarrow$ une information de retour indique une erreur et/ou le log, on ne va pas plus loin
 - ☐ Cela peut être un mail, un SMS, un rapport statique

Maven™npm

Etape - 2 - Compilation

- Il faut donc bien mettre en place la mécanique de build / compilation dans son projet
- L'ordonnanceur respecte vos scripts/outils de build **il ne les remplace pas**
- La mise en place des scripts de build représente ~2j/h par technologie
 - C'est une étape très importante de votre projet

Etape - 3 - Tests Unitaires

➤ L'ordonnanceur va lancer vos tests

- Il peut faire usage de votre outil de build et de vos options de lancement des tests

- ☐ Maven / Gradle / Ant pour le Java
- ☐ NPM pour le JS
- ☐ ...

- Si les tests sont ok $\Leftrightarrow$ passage à l'étape suivante
- Sinon $\Leftrightarrow$ une information de retour indique une erreur et le log de passage des tests, on ne va pas plus loin

The JUnit logo, featuring the word "JUnit" in a stylized font with "J" in green and "Unit" in red.The PHPUnit logo, featuring a blue square with a white circle and a dashed line, and the text "PHPUnit" below it.

Etape - 3 - Tests Unitaires

- Il faut donc écrire des tests unitaires
- **C'est la pierre angulaire** de l'intégration continue
 - sans test = pas d'intégration continue
- Les tests sont Unitaires :
 - ils doivent tester le code de manière courte et simple
 - ils ne sont pas des scénarios fonctionnels,
 - ils n'ont besoin de **RIEN** = ils n'ont pas besoin d'un serveur de base de données, d'un serveur web, ...

Etape - 3 - Tests Unitaires

- L'écriture des tests est très longue
 - ~50% en plus du temps de développement
 - ❑ Si vous avez mis 2 j/h à réaliser une fonctionnalité = il faudra ajouter 1 j/h pour écrire les tests = 3 j/h de charge au total
- Si votre projet est multi-techno
 - Plusieurs framework de tests à maîtriser
- En général on veut **80%** de couverture de code
 - C'est-à-dire que vos tests passent dans 80% de vos lignes de code

Etape - 3 - Tests Unitaires

- Question : je fais comment quand j'ai besoin d'une base données ou d'un serveur web ?
 - Vous pouvez réaliser des mock/bouchons
 - Vous pouvez faire des tests d'intégration (attention vous êtes plus proche du DC)

Etape - 3 - Tests Unitaires

- Le lancement des tests peut nécessiter :
 - l'installation d'une base de données,
 - l'initialisation de la base de données,
 - la mise en place d'un jeu de données,
 - l'installation d'un fichier particulier
 -

- Tous les préparatifs peuvent être gérés par
 - Du code (En Java via Spring par exemple)
 - Des VM (Via docker par exemple)
 - Des éléments déjà installés à la main sur le serveur de build

Etape - 3 - Tests Unitaires

- Par exemple en Docker, juste pour le lancement des tests on peut se retrouver avec :
 - Un docker MySQL
 - Un docker Java / PHP / NodeJS
- Du coup, en plus des 50% de temps associé à l'écriture des tests il faut prévoir leur mise en place
 - Sera en fonction de l'architecture du projet
 - Rôle du devops ou de l'équipe d'intégration

Etape - 4 - Qualité

- L'ordonnanceur va lancer et envoyer votre code à l'outil d'analyse qualité
 - Il peut faire usage de votre outil de build
 - ❑ Maven / Gradle / Ant pour le Java
 - Scriptes pour les autres technologies (ex: Sonar Scanner)
- Si les critères qualités sont ok $\Leftrightarrow$ passage à l'étape suivante
- Sinon $\Leftrightarrow$ une information de retour indique une erreur et le log du passage qualité, on ne va pas plus loin

The Sonarqube logo, featuring the word "sonarqube" in a sans-serif font with a blue wave icon to its right.

Etape - 4 - Qualité

- La qualité est optionnelle mais il est conseillé de la mettre en place
 - Permet de garantir une meilleur maintenabilité du code
- La qualité c'est l'analyse de votre code
 - Est-il bien documenté
 - Est-il bien formaté
 - Respecte il les normes de codages
 - Suit-il les règles des framework (ex: MVC)
 - Qu'elle est votre couverture de tests
- Sonar peut aussi indiquer les problèmes de sécurité ainsi que le non-respect des bonnes pratiques

Etape - 5 - Livrables

- L'ordonnanceur va lancer la fabrication des livrables
 - Il peut faire usage de votre outil de build



WAR,
JAR,
EXE, ...

- Les livrables

- Ce que l'on donne à la production pour que le programme fonctionne
- Ce que l'on donne au client

Etape - 6 - Déploiement

- Sur cette étape, on passe sur du Déploiement Continue
- L'ordonnanceur va
 - Optionnel : créer un environnement de livraison (des Docker pour déployer le projet)
 - Installer / déployer le livrable sur l'environnement de livraison déjà en place
 - ❑ Souvent = faire un copier-coller du livrable pour le placer dans un endroit clef

Etape - 6 - Déploiement

- Dans le cas où c'est l'ordonnanceur qui doit fabriquer l'environnement
 - C'est vous qui lui expliquez comment via des scripts (docker par exemple)
- Cette étape :
 - permet de limiter les erreurs humaines lors du déploiement d'architectures complexes
 - elle peut être jouée plusieurs fois afin de réaliser des environnements différents (test/prex/prod)

Etape - 7 - Tests d'intégrations

- Un test d'intégration implique que le projet est déployé et que tous les éléments externes sont en place
- Si c'est un projet web :
 - Il est sur son serveur web qui est correctement paramétré et lancé
- Si il y a une base de données
 - Elle est installée / initialisée correctement paramétrée

Etape - 7- Tests d'intégrations

- Un test d'intégration se représente souvent sous la forme d'un scénario.

- Par exemple sur un projet web
 - Je vais sur la page login
 - J'indique un bon login/pwd
 - Je me retrouve sur la page menu
 -

Etape - 7 - Tests d'intégrations

- Les scénarios sont réalisés via
 - Des outils : JMeter / Gatling / Selenium / Postman / SoapUI
 - Du Code : via un framework
- L'ordonnanceur va lancer vos tests d'intégrations
 - Il peut faire usage de votre outil de build et de vos options de lancement
- Si les tests sont ok $\Leftrightarrow$ passage à l'étape suivante (souvent une phase de déploiement sur prex/prod)
- Sinon $\Leftrightarrow$ une information de retour indique une erreur et/ou le log, on ne va pas plus loin et on annule la phase de déploiement

A vous de Jouer

- Identifier les outils de build qui vont vous permettre de faire des build
 - Valider / paramétrer vos mécaniques de build
 - Lancer une build en local sur sa machine de dev
- Trouver les framework qui vont vous permettre de faire des tests unitaires
 - Ecrire un test unitaire (pour le moment un seul histoire de 'tester' le framework de test)
 - Lancer son test en local sur sa machine de dev

A vous de Jouer

- Identifier l'ordonnanceur de build
 - Lire sa documentation, regarder le langage de Scripting associé
 - S'inspirer d'un exemple pour le mettre en place sur son projet
 - ❑ Pour le moment partie (build + test uniquement)
- Si aucun, en choisir un (ex: Jenkins) et réaliser les mêmes étapes

A vous de Jouer

- Installer Sonar sur sa machine de dev
- Lancer une analyse qualité sur son projet
- Voir comment scripter le lancement d'analyse qualité
- Ajouter le lancement dans l'ordonnanceur
 - ➡ Ne sera pas toujours possible

A vous de Jouer

- Choisir un outil pour ses tests d'intégrations
- Réaliser au moins un scénario pour ses tests d'intégrations
- Voir comment scripter le lancement des tests d'intégrations
- Ajouter le lancement dans l'ordonnanceur
 - ➡ Ne sera pas toujours possible

Annexes - GitLab

- Comment puis-je enclencher l'IC dans GitLab
 - A la racine de votre projet, il suffit de créer un fichier **.gitlab-ci.yml**
 - Il contiendra toutes les étapes que vous souhaitez lancer lors de votre IC
 - Il sera enclenché **automatiquement** à chaque **push**

Name
database
postman
src
.gitattributes
.gitignore
 .gitlab-ci.yml
CHANGELOG
LICENSE
pom.xml

Annexes - GitLab

- Je le trouve où le résultat de mon IC ?
 - ➡ Sur le serveur GitLab, dans votre projet

The screenshot shows the GitLab interface for the 'ferret.renaud.banque' project. The left sidebar has a 'CI / CD' section with 'Pipelines' selected. The main content area shows a table of pipeline runs. The top row, #235, is highlighted with a red box and shows a 'failed' status. Other rows show 'passed' or 'canceled' statuses.

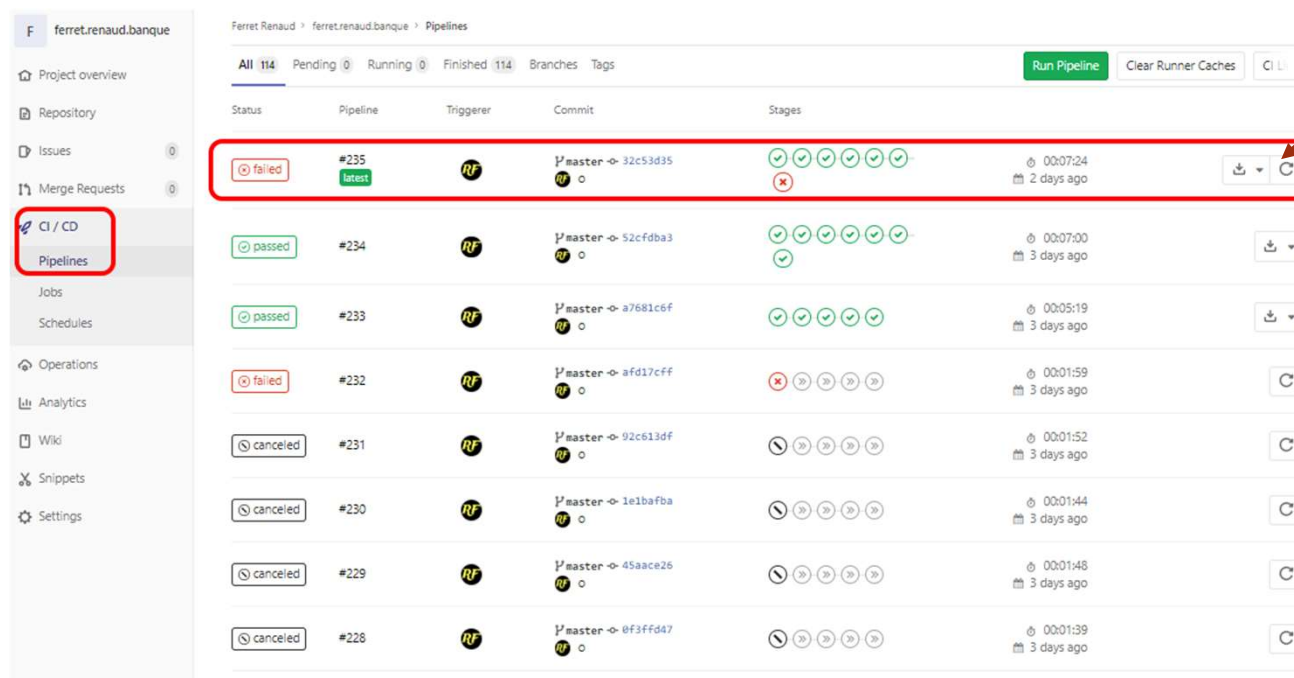
Status	Pipeline	Triggerer	Commit	Stages	Duration	When	Actions
failed	#235 latest	RF	P'master -> 32c53d35	6 stages (all failed)	00:07:24	2 days ago	Download, Refresh
passed	#234	RF	P'master -> 52cfdab3	6 stages (all passed)	00:07:00	3 days ago	Download
passed	#233	RF	P'master -> a7681c6f	6 stages (all passed)	00:05:19	3 days ago	Download
failed	#232	RF	P'master -> afd17cff	6 stages (all failed)	00:01:59	3 days ago	Refresh
canceled	#231	RF	P'master -> 92c613df	6 stages (all canceled)	00:01:52	3 days ago	Refresh
canceled	#230	RF	P'master -> 1e1bafba	6 stages (all canceled)	00:01:44	3 days ago	Refresh
canceled	#229	RF	P'master -> 45aace26	6 stages (all canceled)	00:01:48	3 days ago	Refresh
canceled	#228	RF	P'master -> 0f3ffda7	6 stages (all canceled)	00:01:39	3 days ago	Refresh

- ❑ Si tout est vert, c'est super, sinon, il faut regarder les logs du Jobs

Annexes - GitLab

36

- ➔ Vous pouvez relancer toute l'IC en cliquant sur l'icone 
- ❑ Cela évite de refaire un *add/commit/push* pour rien



Status	Pipeline	Triggerer	Commit	Stages	Duration	Actions
failed	#235 latest	RF	Y'master -> 32c53d35	✓ ✓ ✓ ✓ ✓ ✓ ✗	00:07:24 2 days ago	Download Refresh
passed	#234	RF	Y'master -> 52cfdab3	✓ ✓ ✓ ✓ ✓ ✓ ✓	00:07:00 3 days ago	Download
passed	#233	RF	Y'master -> a7681c6f	✓ ✓ ✓ ✓ ✓ ✓	00:05:19 3 days ago	Download
failed	#232	RF	Y'master -> afd17c6f	✗ ✗ ✗ ✗ ✗ ✗	00:01:59 3 days ago	Refresh
canceled	#231	RF	Y'master -> 92c613df	⏸ ✗ ✗ ✗ ✗	00:01:52 3 days ago	Refresh
canceled	#230	RF	Y'master -> 1e1bafba	⏸ ✗ ✗ ✗ ✗	00:01:44 3 days ago	Refresh
canceled	#229	RF	Y'master -> 45aace26	⏸ ✗ ✗ ✗ ✗	00:01:48 3 days ago	Refresh
canceled	#228	RF	Y'master -> 0f3ffd47	⏸ ✗ ✗ ✗ ✗	00:01:39 3 days ago	Refresh

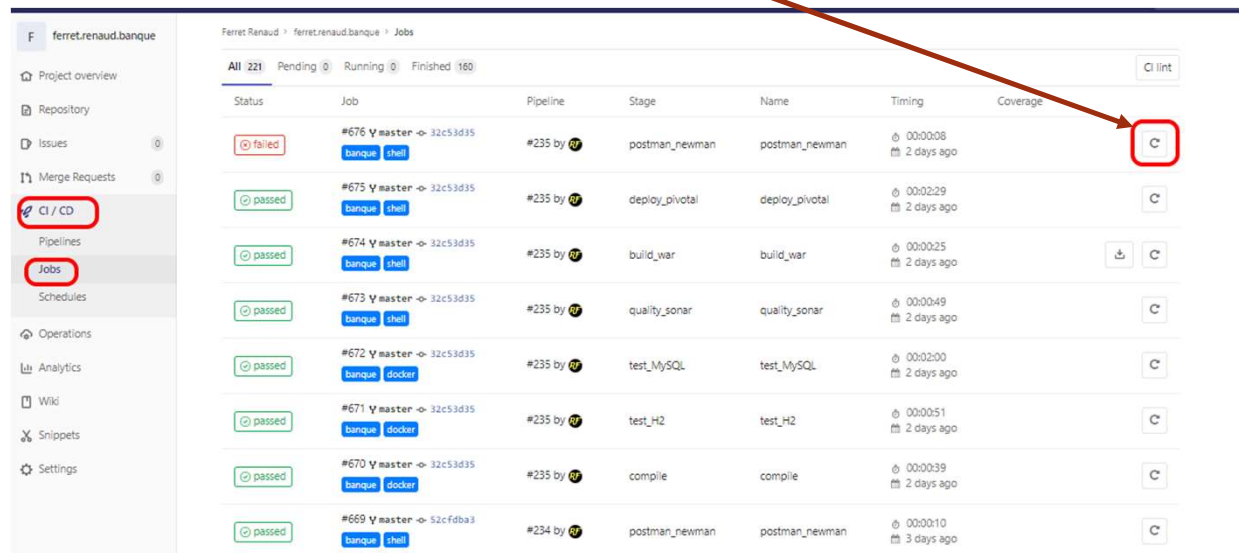
Annexes - GitLab

37

➔ Pour relancer un seul Job,

❑ il suffit de cliquer sur  du job visé, puis de cliquer sur le bouton Retry en haut à gauche

❑ Ou, cliquer dans votre menu CI/CD / Jobs, puis de cibler le job que vous voulez relancer 



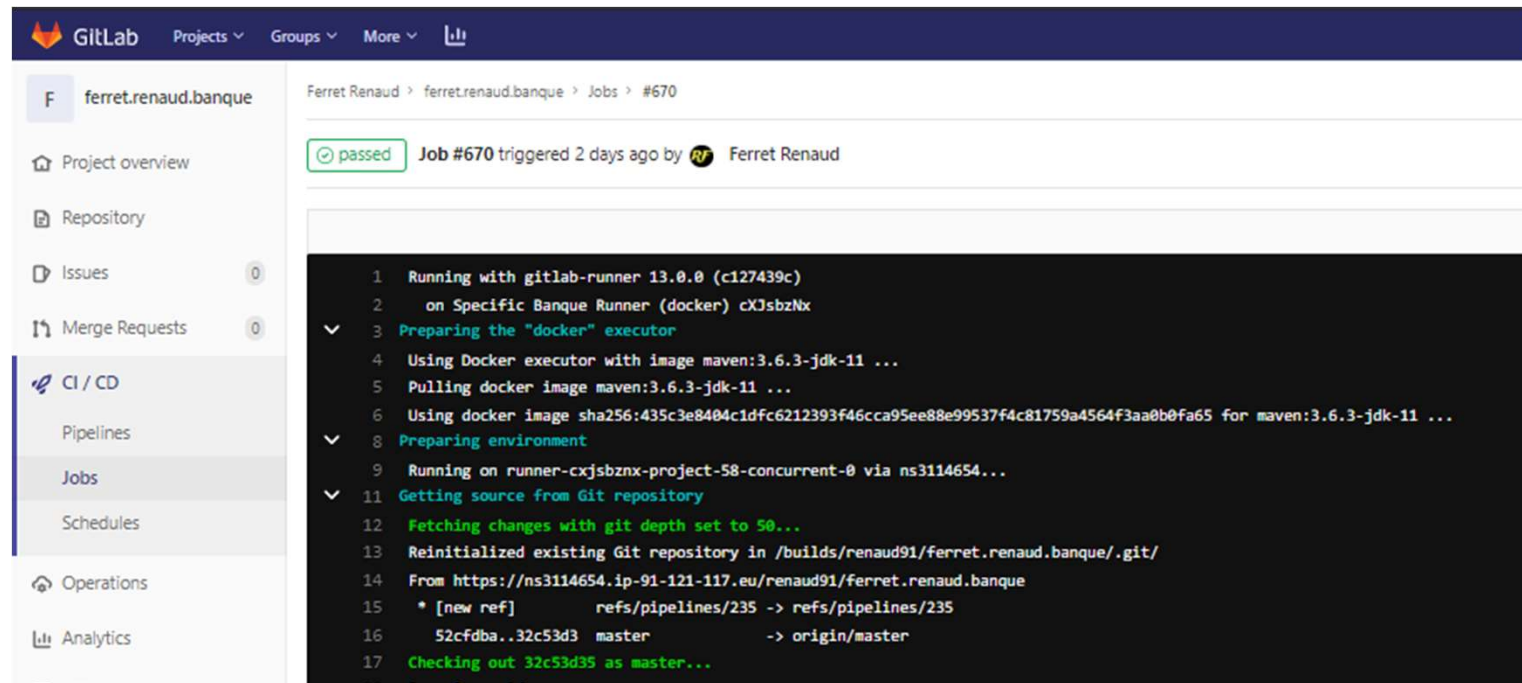
Status	Job	Pipeline	Stage	Name	Timing	Coverage
failed	#676 ✓ master → 32c53d35 banque shell	#235 by 	postman_newman	postman_newman	00:00:08 2 days ago	
passed	#675 ✓ master → 32c53d35 banque shell	#235 by 	deploy_pivotal	deploy_pivotal	00:02:29 2 days ago	
passed	#674 ✓ master → 32c53d35 banque shell	#235 by 	build_war	build_war	00:00:25 2 days ago	 
passed	#673 ✓ master → 32c53d35 banque shell	#235 by 	quality_sonar	quality_sonar	00:00:49 2 days ago	
passed	#672 ✓ master → 32c53d35 banque docker	#235 by 	test_MySQL	test_MySQL	00:02:00 2 days ago	
passed	#671 ✓ master → 32c53d35 banque docker	#235 by 	test_H2	test_H2	00:00:51 2 days ago	
passed	#670 ✓ master → 32c53d35 banque docker	#235 by 	compile	compile	00:00:39 2 days ago	
passed	#669 ✓ master → 52cfdba3 banque shell	#234 by 	postman_newman	postman_newman	00:00:10 3 days ago	

Annexes - GitLab

38

➤ Et les logs, ils sont où ?

➡ Sélectionner un job (directement via les Pipelines ou les Jobs)



The screenshot displays the GitLab web interface. On the left sidebar, the 'CI / CD' section is expanded, showing 'Pipelines', 'Jobs', and 'Schedules'. The 'Jobs' option is selected. The main content area shows the details for 'Job #670' in the 'ferret.renaud.banque' project. The job status is 'passed' and it was triggered 2 days ago by 'Ferret Renaud'. Below this, the job logs are displayed, showing the execution steps and commands.

```
1 Running with gitlab-runner 13.0.0 (c127439c)
2 on Specific Banque Runner (docker) cXJsbzNx
3 Preparing the "docker" executor
4 Using Docker executor with image maven:3.6.3-jdk-11 ...
5 Pulling docker image maven:3.6.3-jdk-11 ...
6 Using docker image sha256:435c3e8404c1dfc6212393f46cca95ee88e99537f4c81759a4564f3aa0b0fa65 for maven:3.6.3-jdk-11 ...
7 Preparing environment
8 Running on runner-cxjsbnx-project-58-concurrent-0 via ns3114654...
9 Getting source from Git repository
10 Fetching changes with git depth set to 50...
11 Reinitialized existing Git repository in /builds/renaud91/ferret.renaud.banque/.git/
12 From https://ns3114654.ip-91-121-117.eu/renaud91/ferret.renaud.banque
13 * [new ref]      refs/pipelines/235 -> refs/pipelines/235
14 52cfdba..32c53d3 master -> origin/master
15 Checking out 32c53d3 as master...
```

Annexes - GitLab

➤ Qu'est ce qu'un Runner dans Gitlab ?

- C'est un processus installé sur la machine qui héberge votre GitLab
- Il a été installé par l'administrateur **du serveur qui héberge GitLab**
 - ❑ Ce n'est pas le programme GitLab qui l'installe, il doit être installé en plus par un administrateur du serveur

➤ A quoi sert un Runner ?

- Un Runner **est responsable** du lancement de votre scripte d'intégration continue

Annexes - GitLab

40

➤ Existe-il plusieurs type de Runner ?

➤ Oui, vous avez les types suivants

❑ **shell** : runner qui sert à lancer des commandes shell

❑ **docker** : runner qui sert à lancer une image docker

❑ **ssh** : runner qui sert à lancer des commandes en ssl

❑ ... : <https://docs.gitlab.com/runner/#selecting-the-executor>

➤ Qui décide du type du Runner et quand ?

➤ L'administrateur de la machine,

➤ Lors de l'installation du Runner

❑ En général, on ne le change pas. Si un veut un autre type de Runner on en déclare un autre

Annexes - GitLab

➤ Comment spécifier le Runner que je souhaite utiliser ?

➤ Au niveau de votre projet, vous avez la liste des Runner dédiés :

❑ En les éditant, vous trouverez leurs *tags*

Tags banque, shell



Auto DevOps can automatically build, test, and deploy applications based on configuration. Learn more about Auto DevOps or use our quick start guide.

Runners

Runners are processes that pick up and execute jobs for GitLab. Here you can manage your runners.

You can set up as many Runners as you need to run your jobs. Runners can be placed on separate users, servers, and even on your local machine.

Each Runner can be in one of the following states:

- active** - Runner is active and can process any new jobs
- paused** - Runner is paused and will not receive any new jobs

To start serving your jobs you can either add specific runners or use Auto DevOps.

Specific Runners

Set up a specific Runner automatically

You can easily install a Runner on a Kubernetes cluster. Learn more about Kubernetes.

- Click the button below to begin the install process by navigating to the Kubernetes page.
- Select an existing Kubernetes cluster or create a new one.
- From the Kubernetes cluster details view, install Runner from the applications list.

[Install Runner on Kubernetes](#)

Set up a specific Runner manually

- Install GitLab Runner
- Specify the following URL during the Runner setup: `https://ns3114654.ip-91-121-117.eu/`
- Use the following registration token during setup: `me2Zv5rKQa-a94L5PKeS`

[Reset runners registration token](#)

- Start the Runner!

Runners activated for this project

Byh4y3pV...	Specific Banque Runner (shell)	banque shell	Pause Remove Runner
cX3sbzNX...	Specific Banque Runner (docker)	banque docker	Pause Remove Runner

Je peux faire usage de ses deux Runner pour ce Projet

Annexes - GitLab

42

- Comment spécifier le Runner que je souhaite utiliser ?
 - Une fois que vous connaissez le/les tags, vous pouvez les indiquer de manière explicite dans votre fichier **.gitlab-ci.yml**

```
compile:  
  stage: compile  
  tags:  
    - banque,shell  
  script:  
    - /usr/bin/mvn clean compile -Dmaven.test.skip=true
```

Annexes - GitLab

43

- Comment spécifier le Runner que je souhaite utiliser ?
 - Et si je ne connais pas mon tags ?
 - ❑ Ne mettez rien, mais **cela ne fonctionnera que si** l'administrateur de la machine a déclaré des Runner sans tags ou communs à plusieurs projets.
- C'est quoi Docker ?
 - C'est un programme qui permet d'exécuter des machines virtuelles
 - ❑ Par exemple, je peux, avec docker
 - lancer un serveur Linux sur une machine Windows **sans installer** de Linux
 - lancer un serveur MySQL sur mon pc **sans installer** MySQL
 - lancer n'importe quoi à partir de n'importe quoi

Annexes - GitLab

44

➤ C'est quoi le rapport entre GitLab et Docker ?

- Docker et Gitlab sont deux programmes/outils totalement différents.

- ☐ Vous pouvez faire du Docker sans faire de GitLab et inversement

- Mais, **dans le cadre de l'intégration continue** vous pouvez rencontrer le besoin d'avoir à votre disposition des programmes (maven, npm, la VM C#, MySQL, ...).

- ☐ Dans ce cas, Gitlab peut lancer des images docker, pour vous, en respectant les contraintes que vous lui aurez données via votre fichier **.gitlab-ci.yml**

- Avantage de cette approche, rien n'est installé sur le serveur GitLab, tout est monté/chargé à la volée
- Inconvénient : il faut que la machine physique qui héberge GitLab soit en mesure de lancer la/les images docker

Annexes - GitLab

45

- Suis-je obligé d'utiliser Docker avec GitLab ?
 - Non : si le/les programmes que vous souhaitez lancer sont déjà installés sur la machine qui héberge votre GitLab
 - Oui : si le/les programmes que vous souhaitez lancer ne sont pas déjà installés sur la machine qui héberge votre GitLab
 - ❑ Ou ne sont pas installables sur la machine qui héberge votre GitLab

Annexes - GitLab

46

- Comment dire à GitLab que l'on veut qu'il utilise une image docker ?
 - Dans votre fichier **.gitlab-ci.yml**, indiquez le mot clef ***image*** suivit du **nom** et du numéro de version de l'image que vous voulez

```
compile:  
  image: maven:3.6.3-jdk-11  
  stage: compile  
  tags:  
    - banque,docker  
  script:  
    - /usr/bin/mvn clean compile -Dmaven.test.skip=true
```

- Pensez à indiquer un *tags* qui cible un Runner de type Docker

Annexes - GitLab

47

➤ Mais je les trouve où ses noms et numéros d'images ?

➡ Sur <https://hub.docker.com/>

